Intelligent Tram Dispatching Under Dynamic Passenger Demand Using Reinforcement Learning

Name	Student Number	Role
Niranjan Sendilkumar	20095917	Participate in Report Writing Writing the DDPG agent code
Neethu Sreekumar	20092130	Participate in Report Writing Writing the TD3 agent code Evaluation and result comparison
Sai Krishna Varshith	20097433	Participate in Report Writing Building the environment Reward and State building
Sudhakar Reddy Mutyam	20099085	Participate in Report Writing Writing the Monte-carlo agent code
Chethan Uma Chowdary	20098860	Participate in Report Writing Writing the Q-learning Agent code

Abstract:

This project addresses the problem of inefficient, fixed-time tram dispatching in urban transit networks by training reinforcement learning (RL) agents to adaptively manage dispatch headways. We design a linear tram route simulation environment in Python featuring five stations, a passenger capacity limit of C = 80, and time-varying Poisson arrival processes representing morning peak, evening peak, and off-peak hours. We implement and evaluate four distinct reinforcement learning algorithms from scratch: Tabular Q-Learning, every-visit Tabular Monte Carlo (MC) Control, Deep Deterministic Policy Gradient (DDPG), and Twin Delayed DDPG (TD3).

Our project trains a reinforcement learning (RL) agent to act as an **automated dispatcher** that dynamically decides whether to dispatch a tram or wait at each 5-minute interval, responding adaptively to real-time commuter demand.

Introduction:

Reinforcement learning (RL) is a branch of machine learning in which an agent learns to make better decisions by interacting with its environment through trial and error. Rather than following a fixed set of predefined rules, the agent observes the current state of the system, chooses an action, and receives feedback in the form of a reward. Over time, it learns which actions produce the best long-term outcomes.

Conventional tram dispatching systems typically rely on fixed timetables or simple rule-based strategies, such as dispatching a tram every 10 minutes or only when passenger queues exceed a certain threshold. Although these methods are straightforward to implement, they struggle to respond to changing passenger demand throughout the day. During busy periods, this can result in overcrowded platforms and long waiting times, while quieter periods may see lightly occupied trams operating unnecessarily, increasing operational costs and reducing efficiency.

Deep reinforcement learning extends traditional RL by combining it with neural networks, enabling the agent to learn from large and complex state spaces that would be difficult to represent using conventional methods. In the context of tram scheduling, this allows the system to consider factors such as vehicle locations, passenger queue lengths, and waiting times simultaneously when making dispatching decisions. This project develops a unified simulation environment to compare the performance of both traditional tabular reinforcement learning algorithms and deep reinforcement learning approaches for adaptive tram dispatching and headway control.

Motivation and Real-World Context

Urban public transit networks are critical to modern cities, influencing economic productivity, carbon emissions, and daily commuter satisfaction. Traditional dispatching systems rely on rigid, fixed-time schedules (e.g., dispatching a tram every 15 minutes) regardless of real-time passenger demand. This rigidity leads to significant inefficiencies: during peak rush hours, station queues overflow, causing long wait times and severe vehicle overcrowding; conversely, during off-peak hours, trams are dispatched empty, resulting in wasted fuel, unnecessary labor, and high wear-and-tear costs. Developing an adaptive, demand-responsive scheduling framework is essential for modern intelligent transportation systems.

Problem Statement

The objective of this project is to design a reinforcement learning agent that adaptively decides when to dispatch a tram from a dispatch depot to minimize passenger waiting times while maintaining low vehicle operating costs. We formulate this problem as an episodic Markov Decision Process (MDP) over an 18-hour operating window (06:00 to 24:00) where:

- The Agent is the transit scheduler at the dispatch terminus (Station 0).
- The Environment is a linear tram network simulator.
- The State Space captures station passenger queues, elapsed dispatch times, and active tram positions.
- The Action Space represents the binary decision to either wait or dispatch a new tram.
- The Reward Function penalizes passenger waiting time, transit travel delay, and tram operating costs, while rewarding successful passenger deliveries.

Why Reinforcement Learning?

The adaptive dispatching problem is characterized by sequential decision-making under uncertainty, where current actions influence future states and rewards. Traditional optimization techniques (like integer programming) struggle to handle the stochastic, non-stationary passenger arrival rates (Poisson peak demands) and require a precise mathematical model of transit dynamics.

Reinforcement learning is well-suited because:

1. Sequential Decisions: RL naturally models the temporal correlation of states, matching how a dispatch decision now impacts passenger accumulation at downstream stations 30 minutes later.

2. Model-Free Learning: Agents learn optimal behaviors through direct interaction with the environment, without requiring pre-defined transition probabilities.

3. Continuous State Processing: Deep RL networks can process continuous queue lengths and tram occupancies directly, avoiding discretization bottlenecks.

Objectives

The aims of this project are to:

1. Build a realistic linear tram network simulator from scratch following a Gymnasium-compatible interface.

2. Implement four RL algorithms from scratch (Tabular Q-learning, Tabular Monte Carlo, DDPG, and TD3) using only NumPy and PyTorch, without pre-built RL libraries.

3. Train all agents under identical demand conditions and evaluate performance using multiple metrics (Wait Time, Occupancy, Dispatches, and Total Rewards).

4. Compare performance against a traditional fixed-time baseline controller.

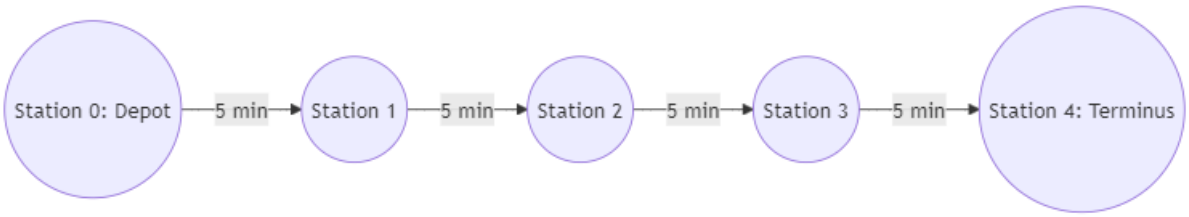
5. Document key challenges (such as credit assignment and overestimation bias) and evaluate tabular vs. deep RL trade-offs.

2.0 Methodology

Environment design:

Linear Route Model

We model a single linear transit line consisting of N = 5 stations (Stations 0, 1, 2, 3, 4). Station 0 serves as the dispatch depot and Terminus A, while Station 4 serves as Terminus B. Trams move uni-directionally from Station 0 to Station 4, taking exactly 5 minutes to travel between adjacent stations. The total travel time from depot to terminus is 20 minutes.



The tram environment is planned to be designed in Python using a custom Gymnasium-compatible structure.
A linear tram route consisting of:

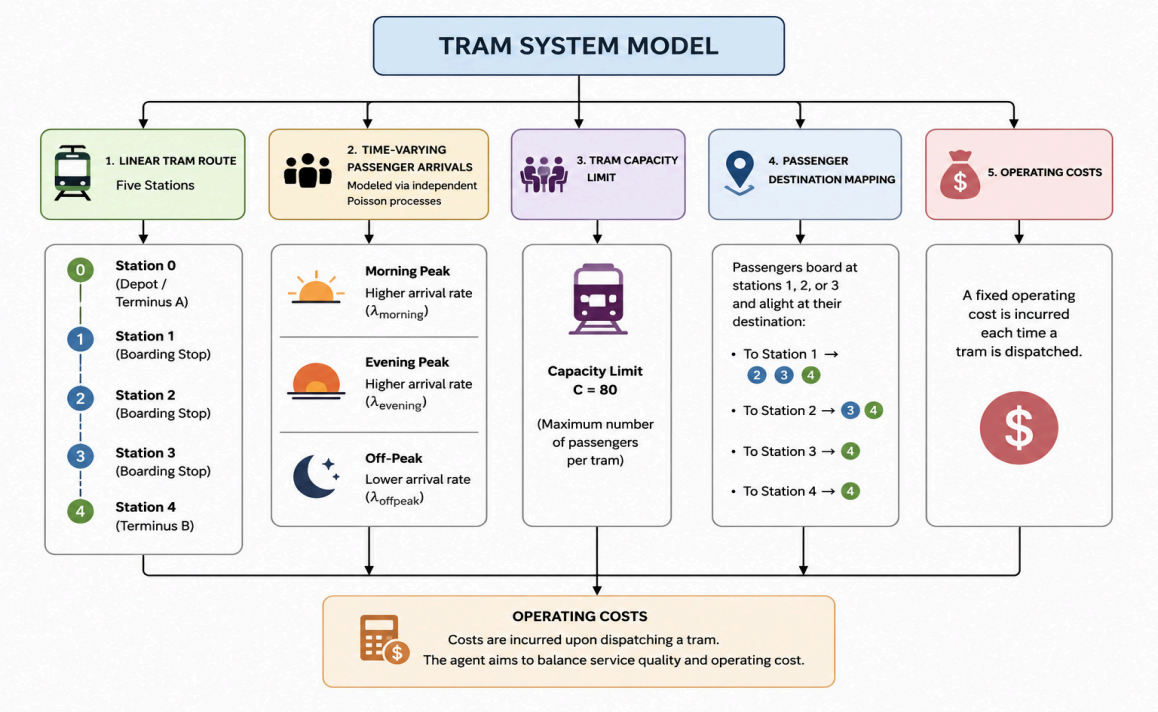
- Five stations:
  - Station 0 (Depot/Terminus A)
  - Stations 1, 2, 3 (boarding stops)
  - Station 4 (Terminus B).
- Time-varying passenger arrivals modeled via independent Poisson processes representing morning peak, evening peak, and off-peak hours.
- A tram capacity limit (C = 80) and passenger destination mapping.
- Operating costs incurred upon dispatching a tram.

Passenger Demand Scenarios:

Passengers arrive at boarding stations (0, 1, 2, 3) according to independent Poisson processes with arrival rates $\lambda_i(t)$ that vary based on the time of day t (minutes since 06:00):

- Morning Peak (07:30 - 09:30): High passenger demand at Station 0 (λ_0 = 1.8 passengers/min) and moderate demand at intermediate stations (λ_1 = 0.8, λ_2 = 0.6, λ_3 = 0.4)
- Evening Peak (16:30 - 18:30): High, uniform demand across all stations (λ_0 = 1.4, λ_1 = 1.0, λ_2 = 0.8, λ_3 = 0.6).
- Off-Peak: Low baseline demand (λ_0 = 0.4, λ_1 = 0.2, λ_2 = 0.2, λ_3 = 0.1).

Passenger destinations are selected uniformly from remaining downstream stations (e.g., a passenger arriving at Station 1 can select Stations 2, 3, or 4 as their destination). Trams have a capacity limit of C = 80. Boarding follows a first-in, first-out (FIFO) queue structure. Passengers alight immediately when the tram reaches their destination.



State space design:

1. **Continuous State Space (Deep RL)**
   The deep reinforcement learning agents (DDPG and TD3) receive a 12-dimensional continuous state vector $S_t \in [0,1]^{12}$:

- q0, q1, q2, q3: Normalized queue sizes at Stations 0, 1, 2, 3 (divided by 100).
- t_day: Normalized time of day (minutes since 06:00, divided by 1080).
- t_elapsed: Normalized elapsed time since last dispatch (divided by 60 mins)
- occ0, occ1, occ2, occ3: Normalized occupancies of trams currently located at Stations 0, 1, 2, 3 (0 if no tram present).
- demand_context: Current demand level (0: Off-Peak, 0.5: Evening Peak, 1.0: Morning Peak).
- N_active: Number of active trams running on the track (divided by 4).

2. **Discrete State Space (Tabular RL)**

To prevent state-space explosion, the tabular agents (Q-learning and Monte Carlo) use a discretized 27-state index representing the cross-product of:

1. Total Queue Size (3 bins): Low (< 10 passengers), Medium (10 - 30 passengers), High (≥ 30 passengers).

2. Demand Context (3 bins): Off-Peak, Evening Peak, Morning Peak.

3. Elapsed Time since last dispatch (3 bins): Short (< 10 mins), Medium (10 - 20 mins), Long (≥ 20 mins).

27{ States} = 3 { Queue Bins} X 3{ Demand Bins} X 3 {Elapsed Bins}

Action space:

Discrete action space has two actions

Action Index	Decision	Description
0	**Wait**	Do not dispatch a tram, wait another 5 minutes.
1	**Dispatch**	Dispatch a new tram from Station 0 and reset the timer.

For the continuous agent (DDPG), the action space is
$a \in [-1,1]$
The environment maps this continuous action using a step-threshold:
$\text{Action}=\{1 \text{ if } a > 0.0$
$\text{Action}=\{0 \text{ if } a \leq 0.0$

Reward function:

The reward function balances passenger wait times, travel delays, transit deliveries, and vehicle operating costs. The reward R_t received at step t is defined as:

Reward = Passenger Delivery Reward−Dispatch Cost−Queue Waiting Penalty−Travel Time Penalty

$$R_t = R_{delivered} . N_{delivered} - C_{Op} . A_t - W_{wait} . \sum_{i=0}^{3} \int_{t}^{t+5} Q_i(\tau)\, d\tau - W_{travel} \sum_{j} \int_{t}^{t+5} P_j(\tau)\, d\tau.$$

where:
$N_{delivered}$ is the number of passengers dropped off at their destinations,
$A_t \in \{0, 1\}$ is the dispatch action,
$Q_i(\tau)$ is the queue size at station i at minute τ ,
$P_j(\tau)$ is the passenger occupancy of tram j at at minute τ

$R_{delivered}$: **+2** for every passenger delivered.

C_{Op}: **−40** whenever a tram is dispatched (operating cost).

W_{wait}: **−0.2** for accumulated passengers waiting at stations.

W_{travel}: **−0.05** for accumulated passenger travel time inside the tram.

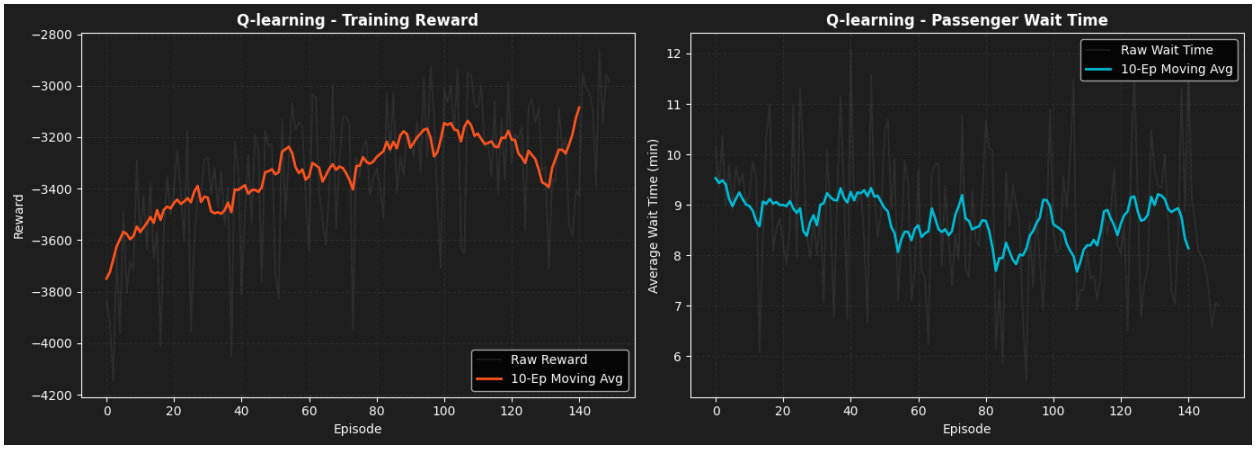
This formulation encourages the RL agent to transport as many passengers as possible while minimizing unnecessary dispatches, passenger waiting times, and overall travel delays.

Algorithms Implemented

1. **Q Learning:**

Q-Learning is an off-policy temporal-difference (TD) algorithm that updates the action-value function Q online at each step using the Bellman optimality equation:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right]$$



2. **Monte Carlo filled Q learning:**

This hybrid tabular method combines the multi-step return estimation of Monte Carlo (MC) methods with the bootstrapping capability of Q-learning. In the initialization phase, the agent executes 100 pure exploration episodes using a random uniform policy. For each episode, we record the visited state-action pairs and compute their total return

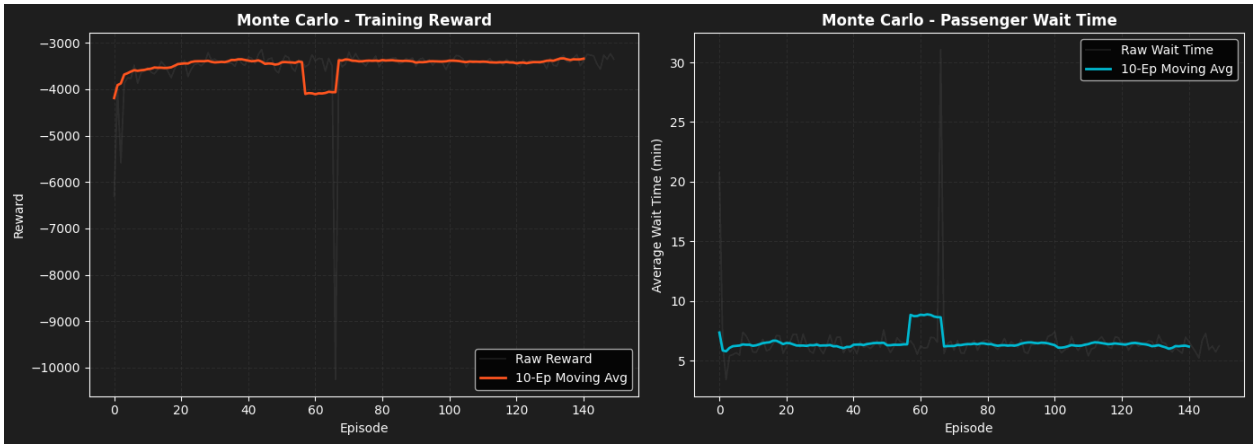
Gt: $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$

The Q-table values are then initialized as the average of all MC returns collected for that state-action pair:

$$Q_{init}(s,a) = \frac{1}{N(s,a)} \sum_{i=1}^{N(s,a)} G_i(s,a)$$

If a state-action pair remains unvisited, it defaults to

0.0. After this initialization phase, the agent trains under standard off-policy Q-learning to refine its policy.



3. Deep Deterministic Policy gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG) is an **actor–critic reinforcement learning algorithm** designed for **continuous action spaces**. Unlike value-based methods such as DQN, DDPG learns both a policy (actor) and a value function (critic), making it well suited for control problems where actions are continuous rather than discrete.

- Actor $\mu(s;\theta^{\mu})$ Outputs a continuous dispatch action a∈[−1,1]
- Critic $Q(s,a;\theta^{Q})$): Estimates the action-value function by taking both the continuous state and action as inputs.
  Targets are updated softly to avoid policy collapse:

$$\theta^{-} \leftarrow \tau\theta + (1-\tau)\theta^{-}$$

4. Twin Delayed DDPG (TD3)

TD3 is an actor-critic algorithm that extends DDPG to prevent the overestimation of Q-values common in actor-critic settings. It introduces three core improvements:

- Twin Critic Networks: Learns two independent critic networks

Qθ1(s,a) and Qθ2(s,a), taking the minimum target value during temporal difference bootstrapping:

$$Y_t = R_{t+1} + \gamma \min_{j=1,2} Q_{\theta_j^-}(S_{t+1}, \tilde{a}_{t+1})$$

- Target Policy Smoothing: Adds clipped Gaussian noise to the target actions to smooth value estimates across actions: $\tilde{a}_{t+1} = \text{clip}\left(\mu_{\phi^-}(S_{t+1}) + \text{clip}(\mathcal{N}(0, \sigma_{smooth}), -c, c), -1.0, 1.0\right)$

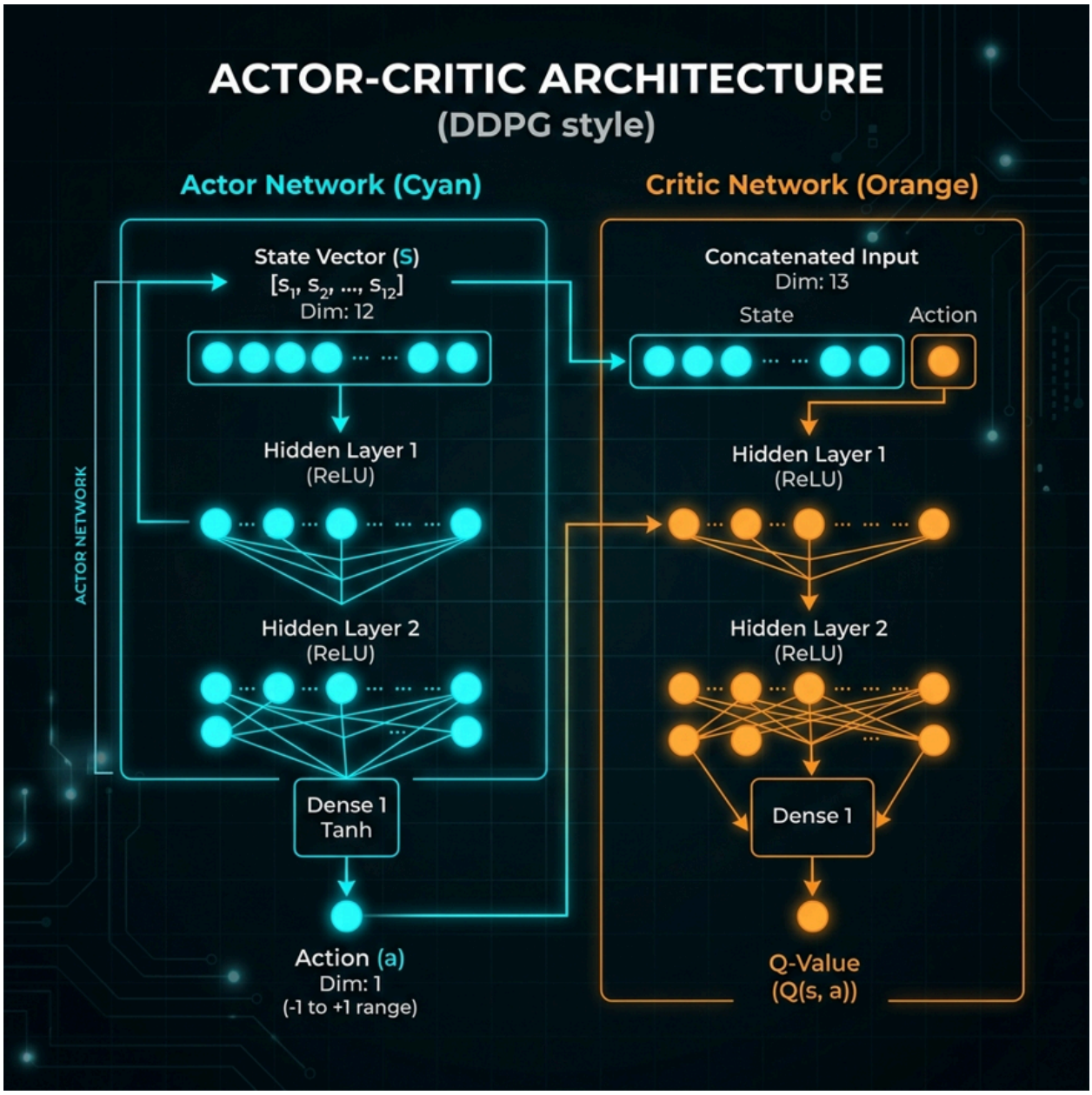
- Delayed Policy Updates: Updates the actor network weights ϕ and all target networks less frequently than the critics (every d steps) to stabilize training

Baseline Fixed-Time Controller:

The baseline represents a traditional urban transit schedule. It dispatches a tram every 15 minutes (3 decision steps) regardless of traffic demand, utilizing an internal timer.

Neural Network Architecture:

To implement the deep reinforcement learning agents (DDPG and TD3), we construct feedforward Multi-Layer Perceptrons (MLPs) in PyTorch. Below is the block diagram showing the data flow, activation functions, and layer dimensions for both the Actor and Critic networks:



Training Protocol:

All models are to be trained under identical conditions:

- Total episodes: 150 (representing 150 days of operations).
- Seeds: Training seeds were controlled to ensure deterministic comparisons.
- Exploration schedule: For tabular agents, ε decayed from 0.50 to 0.02 at a rate of 0.96 per episode. For DDPG and TD3, Gaussian action noise \sigma decayed from 0.30 to 0.05 at a rate of 0.96.
- Hardware: Trained on CPU.

Hyperparameters Table:

Hyperparameter	Q-Learning	Monte Carlo	DDPG	TD3
Discount Factor (γ)	0.99	0.99	0.99	0.99
Learning Rate (α)	0.10	0.10	Actor: 1e-3, Critic: 1e-3	Actor: 1e-3, Critic: 1e-3
Initial Exploration	ε = 0.50	ε = 0.50	Noise σ = 0.30	Noise σ = 0.30
Minimum Exploration	ε = 0.02	ε = 0.02	Noise σ = 0.05	Noise σ = 0.05
Decay Rate (per ep.)	0.96	0.96	0.96	0.96
Replay Buffer Capacity	N/A	N/A	10,000	10,000
Batch Size	N/A	N/A	32	32
Target Soft Update (τ)	N/A	N/A	0.005	0.005
Network Architecture	N/A	N/A	Actor: 12 -> 32 -> 32 -> 1 Critic: 13 -> 32 -> 32 -> 1	Actor: 12 -> 32 -> 32 -> 1 Critics: 13 -> 32 -> 32 -> 1
Policy Delay / Smoothing	N/A	N/A	N/A	Delay: d=2, Noise: 0.20

Ethical considerations:

The deployment of automated scheduling agents in urban transit networks carries significant ethical responsibilities:

- Service Equity and Fairness: Because reinforcement learning agents maximize cumulative reward, they are prone to optimizing headway to favor high-density stations (e.g., Station 0) while neglecting intermediate, low-demand stations (e.g., Station 3). If a tram is constantly dispatched full from Station 0, it leaves no capacity for passengers waiting downstream, effectively denying service to commuters at minor stations. To address this, our reward function includes an explicit waiting-time penalty weight (W_{wait} = 0.2) applied across all stations, ensuring the agent is penalized for neglecting any segment of the population.
- Accessibility and Comfort: Overcrowding inside vehicles (C=80 capacity limit) must be monitored to ensure passenger comfort and compliance with safety regulations, avoiding situations where safety is compromised for operational profit.

Risk Assessment:

Developing and deploying RL controllers from scratch involves several technical and operational risks:

1. State-Space Explosion: Tabular methods (Q-Learning, Monte Carlo) are highly susceptible to the curse of dimensionality. Discretizing a 12-dimensional state vector directly would result in millions of states, causing memory overload and failure to converge. We mitigate this by grouping features into a coarse 27-state index.

2. Convergence Instability: Deep RL algorithms (DDPG) are prone to policy collapse and action saturation due to value overestimation. We address this risk by upgrading the pipeline to TD3, which stabilizes training through twin critics and target action smoothing.

3. Operational Failure Mode (Deadlocks): If the agent learns a sub-optimal policy, it might stop dispatching entirely to avoid operating cost penalties (C_{Op} = 40), causing system-wide transit paralysis. We mitigate this by ensuring a diverse exploration noise schedule (σ decaying from 0.30 to 0.05) to prevent early local minima trapping.

References

[1]
T. Emunds and N. Nießen, “Utilizing phase-type distributions for queueing-based railway junction performance determination,” *Journal of Rail Transport Planning and Management*, vol. 35, Sep. 2025, doi: 10.1016/j.jrtpm.2025.100523.

[2]
Q. Qu *et al.*, “An Improved Reinforcement Learning Algorithm for Learning to Branch,” Jan. 2022.

[3]
R. Dhiman, Sushma, N. T. Singh, and R. Grover, “Metro Route Optimization for Urban Public Transportation Using Q-Learning,” in *IEEE International Conference on Recent Advances in Science and Engineering Technology, ICRASET 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/ICRASET63057.2024.10895966.

[4]
B. Su, Z. Wang, S. Su, and T. Tang, “Metro Train Timetable Rescheduling Based on Q-learning Approach,” in *2020 IEEE 23rd International Conference on Intelligent Transportation Systems, ITSC 2020*, Institute of Electrical and Electronics Engineers Inc., Sep. 2020. doi: 10.1109/ITSC45102.2020.9294514.

[5]
P. Yue, Y. Jin, X. Dai, Z. Feng, and D. Cui, “Reinforcement Learning for Online Dispatching Policy in Real-Time Train Timetable Rescheduling,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, pp. 478–490, Jan. 2024, doi: 10.1109/TITS.2023.3305074.